QualityForge 全项目产品需求文档

AI 原生端到端测试平台 | 项目级 PRD（模块详细设计版）

文档类型	版本	状态	日期
项目级产品需求文档（PRD）	v1.1	评审基线候选	非开发批准	2026-08-10

文档定位

本文件覆盖 QualityForge 全平台愿景、用户、九大模块与平台设置、核心流程、AI 治理、数据与接口依赖、非功能要求、版本路线和验收门禁。它是便于统一评审的整合 PRD，不替代术语、领域模型、OpenAPI、DB 和 AI 治理等专项权威源。

0. 文档控制与阅读说明

项目	内容
产品	QualityForge AI 原生端到端测试平台
文档状态	整合评审基线候选；当前项目仍处于纯文档/设计阶段
目标读者	产品负责人、测试负责人、研发、架构、安全、运维、AI 工程与验收人员
规范依据	ISO/IEC/IEEE 29148 需求结构、通用产品 PRD、Given/When/Then 验收表达
范围	全平台产品需求；具体线上字段与响应以 OpenAPI 为准
本版修订	细化 10 个功能模块、58 个子模块，补充业务痛点、设计思路、功能边界、设计理由及上下游依赖

0.1 目录

- 1 产品摘要与商业问题
- 2 用户、角色与权限
- 3 产品范围与成功标准
- 4 全平台信息架构
- 5 核心端到端流程

- 6 模块功能需求
- 7 AI、Agent 与 Skill 治理
- 8 数据、集成与接口需求
- 9 非功能需求
- 10 路线图与版本范围
- 11 验收与发布门禁
- 12 风险、依赖与未决事项
- 附录 权威源与追踪规则

1. 产品摘要与商业问题

1.1 产品定位

QualityForge 是面向测试工程师和质量团队的 AI 原生端到端测试平台。平台将 AI 嵌入需求理解、测试设计、资产管理、执行、失败分析、知识沉淀与 CI/CD 门禁，而不是把 AI 限定为独立聊天窗口。核心价值不是生成更多文本，而是建立从来源到结论、从设计到执行、从失败到知识的可审计闭环。

1.2 待解决的问题

- 需求、接口契约、测试用例、脚本、数据和执行证据分散，跨阶段追踪成本高。
- 测试设计依赖个人经验，边界、异常、权限和跨系统链路容易遗漏。
- 自动化资产维护成本高，Locator 漂移、数据准备和环境差异反复消耗人力。
- 失败排查依赖人工拼接日志、Trace、截图和变更，重复问题无法形成组织知识。
- CI 回归范围粗放，质量门禁缺少统一、可解释和可重放的判断依据。
- AI 输出若缺少来源、外部 oracle、人工审批和最小权限，会放大错误而非降低风险。

1.3 产品原则

原则	产品要求
可信链优先	RequirementDocument → TestingIR → BusinessRule → Scenario → DesignTestCase → Assertion → Coverage 全链可追溯。
人类最终负责	关键资产发布、外部写入、门禁放行和高风险变更必须由有权用户确认。
默认安全	AI 默认最小权限、零网络、无 Secret、fail-closed；激进自治只能显式启用。
渐进交付	先验证单工作流和单节点闭环，再引入分布式、多 Agent、市场和高级预测。
契约先行	术语、状态、实体、API、DB 与验收标准在开发前完成同步和复审。

2. 用户、角色与权限

现行总纲中的“用户只有一个测试工程师”不能覆盖审批、租户、平台管理、Runner 运维和发布门禁。项目级 PRD采用多角色模型；一个人可以兼任多个角色，但权限必须按职责授予。

角色	主要目标	关键权限边界
测试工程师	设计、维护和执行测试	可生成草稿和发起执行；不能自行批准需独立复核的 AI 产物
测试负责人/审批人	评审正式资产与质量门禁	批准/驳回 DesignTestCase、ReviewTask、Coverage 与门禁例外
自动化工程师	维护脚本、Runner 和执行能力	不得读取未授权租户 Secret；不决定发布放行
研发工程师	查看失败证据与修复建议	默认只读质量与执行结果，不管理平台密钥
发布负责人	基于证据作发布决策	可处理门禁例外；AI 只能建议，不能代签
租户/平台管理员	管理用户、模型、Runner、Secret 和集成	高风险配置需审计；密钥不可回显
AI/Skill 管理员	管理 Agent、Prompt、Skill 和评测	发布能力需准入评审，不能批准其自产业务资产

3. 产品范围与成功标准

3.1 产品边界

范围内	范围外或延后
需求分析、测试设计、资产、执行、质量分析、Skill、Agent、知识、CI/CD、平台设置	通用项目管理、缺陷管理全量替代、代码托管、完整 CI 引擎
API 与 Web UI 测试为首要执行类型	移动真机云、全协议性能平台、完整安全扫描平台在首版不自研
与 Jira/Git/CI/通知工具集成	未经授权直接写外部系统或生产环境
AI 辅助、确定性校验、独立复核与人工审批	无人监督的生产自治和模型自评即通过

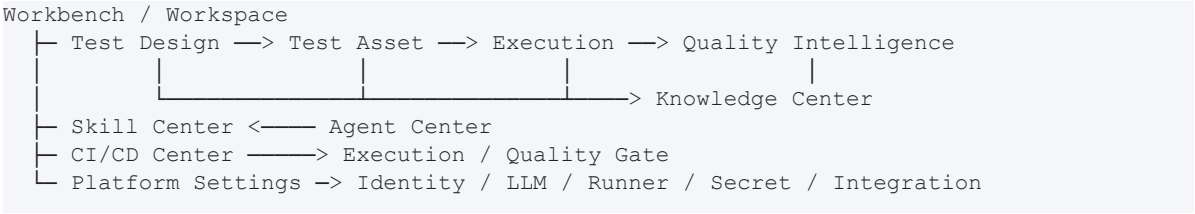
3.2 成功指标

指标	定义	当前状态	验收要求
测试设计耗时降低	同类需求从接收到已批准用例的人员工时变化	目标曾写 80%，缺基线	建立对照任务、样本量和人工修改口径后冻结目标
失败排查耗时降低	从失败产生到确认根因分类的中位/P95 时间	目标曾写 80%，缺基线	以人工确认结果为 oracle，分类型统计
知识检索相关性	Top-K 结果中经人工判定相关的比例	目标曾写 95%，缺评测集	建立标注查询集和 Recall/Precision 指标
CI 回归时间降低	相同风险覆盖约束下 TIA 前后执行时长变化	目标曾写 60%，缺对照	同时报告漏选率，不能只优化速度
可信链完整率	关键资产具有有效 sourceTrace 的比例	新增核心指标	Gate 0 关键链目标 100%
AI 人工修改率	AI 草稿获批前发生实质修改的比例	待采集	按 Skill/版本/风险等级持续观测

指标纪律

未完成基线、测量方法、样本和 oracle 前，80%/95%/60% 等只能视为假设，不得写入开发完成或上线验收结论。

4. 全平台信息架构



导航域	主要页面
Workbench	全局任务、风险、待评审、执行进度和 AI 建议入口
Workspace	空间列表、创建向导、环境、Secret、OpenAPI 导入、扫描进度
Test Design	需求库、质量评审、IR/规则、方案、场景树、用例评审板
Test Asset	用例库、套件、断言、Locator、数据、契约、Mock
Execution	执行计划、任务控制台、Runner、日志/证据
Quality	报告、失败分析、聚类、Coverage、风险
AI 能力	Skill、Agent、Prompt、Workflow、Trace、评测
Knowledge / CI / Settings	知识与基线、流水线与门禁、租户级配置

5. 核心端到端流程

5.1 从需求到正式测试资产

- 创建 Workspace，登记被测系统、环境、权限和来源资料。
- 导入 RequirementDocument，保留原文件、版本、校验和来源。
- 执行需求质量评审；REJECT 停止，PASS_WITH_REVIEW 创建 ReviewTask，PASS 才能自动继续。
- 编译并独立验证 TestingIR，提取 BusinessRule、流程、角色、状态和 SourceTrace。
- 生成 TestDesignPlan、TestScenario、DesignTestCase 与候选 Assertion。
- 独立验证断言和 Coverage；测试负责人审批后将设计用例迁移为正式 TestCase。

5.2 从测试资产到质量决策

- 把正式 TestCase 组成 TestSuite 或 ExecutionPlan，并选择环境与 Runner 能力。
- 任务执行时通过 REST 获取状态，通过 WebSocket观察日志、进度和诊断事件。

- ExecutionResult 固化步骤、断言、日志、Trace、截图、耗时及错误证据。
- Quality Intelligence 生成统计、Coverage、候选 RCA 和失败聚类；人工确认关键结论。
- 经审核的解决模式进入 Knowledge Center；CI/CD 依据版本化 QualityGate 计算结果。

5.3 Agent/Skill Gate 0 流程

```
Supervisor 预检 → Requirement Quality Gate → 规则/领域/状态提取
→ Requirement Compiler → Independent IR Validator → 策略/场景/用例/断言生成
→ Assertion Verifier → Coverage Analyzer → HITL Gatekeeper → 人工决定
```

6. 模块功能需求

本章不仅列出平台能力，还解释每个模块为什么存在、如何拆分、各子模块承担什么职责，以及它们如何衔接。字段、完整状态机和具体请求响应仍以专项权威源为准；优先级表示产品路线，不表示当前已实现。

6.1 Workspace（测试工作空间）

全平台顶层业务容器和 AI 上下文边界，承载被测系统、环境、凭证及初始化扫描。

阶段	主要用户	核心对象	完成结果
V1	测试工程师、空间负责人、平台管理员	Workspace、WorkspaceEnvironment、WorkspaceSecret、WorkspaceAIContext	所有下游资产均绑定 workspaceId/tenantId；无权跨空间读取或执行。

6.1.1 业务痛点

- 被测系统、仓库、接口契约、环境地址和测试凭证分散在个人脚本或 CI 变量中，无法形成统一上下文。
- Dev/Test/Staging 配置容易漂移，同一用例换环境后经常需要手工改 URL、账号和变量。
- AI 生成测试时缺少技术栈、系统边界和契约信息，只能依赖用户反复补充上下文。
- 删除项目或共享凭证时缺少清晰的归属与审计边界，容易误删、越权或泄露。

6.1.2 设计思路与关键取舍

- 以 Workspace 作为租户内的最小业务隔离和上下文容器，所有需求、资产、执行和知识必须归属一个 Workspace。
- 把普通环境配置与 Secret 引用分离：业务模块只看到变量名和 SecretRef，明文只在受控执行时短暂注入。
- 把 AI 扫描设计为异步、可观察、可重放的初始化任务，原始来源与扫描产物同时保存，避免把模型摘要当成原始事实。
- 默认归档而不是立即物理删除；跨模块清理采用显式确认和补偿流程，防止级联误删。

6.1.3 子模块详解

子模块 1 | 空间生命周期管理

目标：建立全平台资源的归属、协作和生命周期边界。

主要功能

- 创建空间并设置名称、描述、负责人和默认策略；校验租户内名称与权限。
- 按成员、状态、更新时间和健康度查询空间列表；展示待评审、失败任务和风险摘要。
- 修改负责人、成员、通知和默认环境；关键权限变更写入审计日志。
- 支持 Active/Archived 等生命周期；归档后禁止新增业务资产，恢复需有权用户操作。

设计理由：解决项目资料无归属、成员权限不清和删除风险；采用顶层容器是因为所有下游模块都需要同一租户与空间边界。

子模块 2 | 被测系统元数据

目标：用结构化资料描述被测系统，使测试设计和执行共享同一事实背景。

主要功能

- 维护业务域、系统类型、前后端技术栈、数据库、仓库和默认分支。
- 登记服务/API 根地址、协议类型、OpenAPI 来源和系统依赖。
- 保存元数据版本及变更人；变更后标记可能受影响的测试设计和执行配置。
- 向 Test Design、Skill 路由和 QI Prompt 提供受控上下文，而非直接拼接全部原文。

设计理由：解决 AI 与测试人员反复补背景的问题；结构化元数据比自由文本更容易校验、检索和驱动自动路由。

子模块 3 | 多环境与凭证管理

目标：让同一测试资产可以安全、可控地运行在多个环境。

主要功能

- 建立 Dev/Test/Staging 等环境，维护 Base URL、区域、变量和启停状态。
- 环境变量支持覆盖关系和差异查看，执行前生成不可变配置快照。
- 凭证只保存 SecretRef、用途和作用域；明文双信封加密，读取接口不回显。
- 执行时按 Workspace、环境、Skill 和任务权限临时注入，日志统一脱敏。

设计理由：解决脚本写死环境与凭证泄露；配置和 Secret 分离可以同时支持复用、轮转和最小暴露。

子模块 4 | AI 上下文与初始化扫描

目标：把契约和系统资料转为可追踪的 AI 上下文，而不是一次性聊天输入。

主要功能

- 支持上传或通过 URL 导入 OpenAPI/Swagger，并执行格式、大小、来源和安全检查。
- 异步解析端点、Schema、依赖与技术栈，持续推送进度、告警和失败原因。
- 生成 WorkspaceAIContext 及候选知识条目，保留 sourceTrace、版本和输入哈希。
- 重新扫描生成新版本；差异需人工确认后才影响正式测试设计。

设计理由：解决 AI 无背景和契约变更不可追踪；异步版本化扫描可以隔离失败，也避免模型结果静默覆盖旧上下文。

6.1.4 输入、输出与依赖

关系	内容
输入	系统资料、OpenAPI、环境参数、成员与授权
输出	Workspace、环境快照、SecretRef、WorkspaceAIContext
上游	Platform Settings 的身份、Secret、集成与 LLM 网关
下游	Test Design、Test Asset、Execution、Knowledge、CI/CD

6.1.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-WS-001	P0	空间生命周期	创建、查询、配置、归档工作空间，并展示空间健康与近期风险。
QF-FR-WS-002	P0	系统元数据	登记技术栈、被测系统基本信息和接入端点。
QF-FR-WS-003	P0	环境与凭证	维护 Dev/Test/Staging 等环境、变量映射和加密 Secret。
QF-FR-WS-004	P1	AI 初始化扫描	导入 OpenAPI/Swagger，构建可追溯的系统上下文与扫描进度。

6.2 Test Design（测试设计）

把需求、接口契约和变更信息编译为可追溯 Testing IR、策略、场景及设计阶段用例。

阶段	主要用户	核心对象	完成结果
V1 核心	测试工程师、需求评审人、测试负责人	RequirementDocument、TestingIR、BusinessRule、TestDesignPlan、TestScenario、DesignTestCase、Assertion、ReviewTask	需求到正式 TestCase 的可信链可追溯；REJECT/BLOCK_HUMAN 时不得自动前进。

6.2.1 业务痛点

- 需求文档非结构化且来源冲突，测试工程师需要手工提炼角色、规则、状态和验收条件。
- 直接让模型生成用例会放大歧义与遗漏，且难以回答某条用例为什么存在、依据来自哪里。
- 测试方案、场景、用例和断言常被混在一个生成动作中，评审粒度过粗。
- 设计草稿与正式用例状态混淆，AI 产物容易绕过独立验证和人工审批。
- 覆盖率缺少明确分母，模型自称“已覆盖”无法作为质量门禁。

6.2.2 设计思路与关键取舍

- 采用类似编译器的分阶段流水线：来源 → 质量 Gate → Testing IR → 规则/策略 → 场景 → 设计用例 → 断言 → Coverage。
- 每个阶段都保存结构化产物、版本、sourceTrace 和裁决；任一硬 Gate 非 PASS 时停止后续生成。
- DesignTestCase 与正式 TestCase 分表、分状态机，通过人工审批执行迁移，保证草稿不会被执行模块误用。
- 生成与验证职责分离：生成 Agent 不得调用独立验证器批准自己的结果，关键问题路由 ReviewTask。
- Coverage 由确定性算法基于需求节点、业务规则、风险和接口计算，AI 只负责解释缺口和建议补充。

6.2.3 子模块详解

子模块 1 | 需求源与版本管理

目标：管理进入测试设计流程的原始资料及其版本关系。

主要功能

- 上传 PDF、Word、Markdown、OpenAPI 或粘贴内容，记录文件名、哈希、来源、版本和解析状态。
- 支持同一需求的新版本导入与差异识别，标记新增、删除、修改和来源冲突。
- 原文只读保存；解析结果和 AI 解释另存，不能回写篡改原始需求。
- 解析失败、格式不支持或内容不完整时输出明确错误与重试入口。

设计理由：解决需求版本混乱和模型改写原文的问题；原始来源与派生产物分离是后续追踪和争议处理的基础。

子模块 2 | 需求质量评审与门禁

目标：在生成 Testing IR 之前判断需求是否足够明确、可测试和一致。

主要功能

- 从歧义、一致性、可测试性、完整性、NFR 可度量性、角色权限六维产生 Finding。
- Finding 必须包含级别、来源位置、证据、原因、建议和待澄清问题。
- 确定性规则处理跨源冲突、不可测验收、不可度量 NFR 和权限矛盾；AI 不得发明业务事实。
- 输出 PASS、PASS_WITH_REVIEW 或 REJECT；后两者创建或关联 ReviewTask。

设计理由：解决缺陷到用例阶段才暴露的问题；先做质量 Gate 比生成后返工成本更低，也能阻止垃圾输入进入可信链。

子模块 3 | Testing IR 与业务规则

目标：把已通过门禁的需求编译成稳定、可验证的中间表示。

主要功能

- 提取角色、业务实体、流程、状态、约束、验收条件、异常和 NFR。
- 生成 BusinessRule，包含 ruleId、条件、动作、优先级、来源和置信度。
- 由独立 IR Validator 校验 Schema、引用、状态可达性、规则冲突和 sourceTrace 完整率。
- 重编译产生新版本并保留旧产物；下游必须锁定使用的 IR 版本。

设计理由：解决自然语言无法稳定驱动多阶段生成的问题；IR 作为编译边界能隔离模型变化，并为确定性验证提供结构。

子模块 4 | 测试方案与策略

目标：根据需求风险、系统技术栈和质量目标定义测试范围与方法。

主要功能

- 选择功能、接口、UI、性能、安全、兼容性等测试类型及优先级。
- 标记范围内/范围外、前置条件、环境、数据、风险和停止条件。
- 引用 Workspace 技术栈、历史缺陷和知识模式，但必须显示来源和版本。
- 方案需关联 RequirementDocument/TestingIR，避免成为独立无来源文档。

设计理由：解决测试方法靠个人临场决定的问题；把策略放在场景前可以先确认测试目标，再批量展开用例。

子模块 5 | 测试场景与业务路径

目标：把业务规则和流程展开为可评审的正向、逆向和异常路径。

主要功能

- 构建主流程、分支、异常、边界、权限和跨系统场景。
- 每个场景声明风险、前置、触发、期望业务结果和上游规则引用。
- 识别重复场景、不可达路径和缺失依赖；允许人工合并、拆分和锁定。
- 场景状态和依赖关系变更后，标记受影响的设计用例待重新生成或复验。

设计理由：解决模型一次生成大量平铺用例而缺少业务结构的问题；先评审场景树能用更低成本发现范围遗漏。

子模块 6 | 设计用例、断言、覆盖与评审

目标：形成可迁移为正式资产的用例草稿，并证明关键要求得到覆盖。

主要功能

- 生成 DesignTestCase 的前置、步骤、数据、预期、优先级、类型和 sourceTrace。
- 为关键步骤生成候选 Assertion，区分状态码、Schema、业务字段、数据库和 UI 断言。
- 独立验证断言是否可观察、可判定、与来源一致，禁止用模型自评代替验证。
- 计算需求、规则、场景、接口和风险覆盖，列出缺口而非只给百分比。
- 人工批准后按迁移规则创建正式 TestCase；驳回或修改均保留版本和审批记录。

设计理由：解决用例不可执行、断言空泛和覆盖不可解释的问题；把验证、Coverage 与人工审批设为最后三道门。

6.2.4 输入、输出与依赖

关系	内容
输入	RequirementDocument、OpenAPI、WorkspaceAIContext、Knowledge Pattern
输出	TestingIR、BusinessRule、TestDesignPlan、TestScenario、DesignTestCase、Assertion、Coverage、ReviewTask
上游	Workspace、Knowledge、AI 治理与 Agent/Skill Gate 0
下游	Test Asset 正式入库、Execution、Quality Intelligence

6.2.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-TD-001	P0	需求导入与解析	导入 PDF、Word、Markdown、OpenAPI 等资料，保留版本和来源。
QF-FR-TD-002	P0	需求质量门禁	从歧义、一致性、可测试性、完整性、NFR、角色权限六维形成证据化裁决。
QF-FR-TD-003	P0	Testing IR 与业务规则	形成结构化 IR、业务规则、Source Trace，并由独立验证器校验。
QF-FR-TD-004	P0	策略与场景生成	依据风险、技术栈和业务流生成测试设计方案与正反场景。
QF-FR-TD-005	P0	用例与断言评审	生成 DesignTestCase 和候选 Assertion，经人工批准后转为正式 TestCase。
QF-FR-TD-006	P1	覆盖率审计	按需求节点、规则、接口和风险维度识别缺口，不以模型自评替代验证。

6.3 Test Asset（测试资产）

集中管理正式测试用例、套件、定位符、测试数据、接口契约和 Mock。

阶段	主要用户	核心对象	完成结果
V1	测试工程师、自动化工程师、数据管理员	TestCase、TestSuite、Assertion、Locator、TestDataProfile、APIContract、MockConfig	正式资产有版本、来源和归属；AI 建议不得绕过人工批准直接覆盖资产。

6.3.1 业务痛点

- 测试用例、脚本、数据、定位符和 Mock 分散维护，修改后无法判断哪些执行计划受到影响。
- 重复或过时用例长期堆积，缺少版本和评审记录，团队不敢删除也无法放心复用。
- UI 定位符脆弱，自动自愈若直接覆盖正式资产会制造更隐蔽的误操作。
- 测试数据可能包含个人信息或生产数据，缺少分类、脱敏和使用范围。
- 接口契约变化与用例、Mock 不联动，漂移往往到执行失败后才发现。

6.3.2 设计思路与关键取舍

- 采用资产图而不是单一用例表：TestCase 关联版本、断言、套件、脚本、定位符、数据和契约。
- 正式资产以不可变版本为主，编辑产生新版本；执行任务锁定快照，避免运行中被修改。
- AI 去重、自愈和数据合成都先生成候选变更，通过影响分析和人工审批后生效。
- 契约、数据和 Mock 作为一等资产管理，使接口测试能够从 Schema 到执行结果闭环。

6.3.3 子模块详解

子模块 1 | 测试用例与版本

目标：管理经批准的正式 TestCase 及其演进。

主要功能

- 支持创建、查询、编辑、归档、复制、导入导出和版本比较。
- 记录前置、步骤、预期、优先级、标签、来源、自动化状态和审批信息。
- AI 规范性检查与去重只给建议，合并必须保留来源和历史引用。
- 被执行计划引用的版本不可原地覆盖；归档前显示影响范围。

设计理由：解决用例库可信度低和改动不可追踪；版本快照让执行结果能够精确回溯到当时的用例定义。

子模块 2 | 测试套件与组织

目标：按业务、风险和执行目的组织可复用测试集合。

主要功能

- 创建静态套件和基于标签/条件的动态套件。
- 配置顺序、并行、失败继续、前后置钩子和环境适用范围。
- 预览实际纳入的用例版本、重复项和依赖冲突。
- 套件变更不自动改变历史执行计划快照。

设计理由：解决每次执行手工选用例和集合不可复现；套件把资产组织与具体执行调度解耦。

子模块 3 | 断言资产

目标：把“预期结果”转成可执行、可验证、可复用的判定规则。

主要功能

- 维护断言目标、表达式、期望值、容差、严重级别和来源。
- 支持 API、Schema、数据库、UI、性能等断言类型。
- 记录 VERIFIED/REJECTED/UNVERIFIED 等验证状态和验证器版本。
- 修改共享断言前显示引用用例与潜在门禁影响。

设计理由：解决自然语言预期无法自动判断和断言重复编写；结构化断言是执行结果可信的最小单元。

子模块 4 | Locator 与自愈

目标：集中管理 UI 元素定位策略和受控自愈候选。

主要功能

- 按页面和逻辑元素维护主定位符、备选定位符、权重和历史成功率。
- 执行失败时收集 DOM、截图和候选 Locator，不在运行中直接覆盖正式资产。
- 候选变更需验证唯一性、稳定性和影响用例，经审批后提升为主定位符。
- 保留 before/after、触发任务和回滚版本。

设计理由：解决 Locator 漂移造成大面积脚本失败，同时避免“自愈成功但点错元素”的静默风险。

子模块 5 | 测试数据与脱敏

目标：提供可复现且合规的测试数据集。

主要功能

- 导入 CSV/JSON 或定义生成规则、字段约束和关联关系。
- 按环境和用途维护数据集版本、有效期和负责人。
- 对个人信息和 Secret 做分类、掩码、合成替代和访问审计。
- 执行时按参数化策略生成数据快照，完成后执行清理或失效。

设计理由：解决测试数据不可复现与隐私风险；将数据配置化可同时支持参数化、并发和清理。

子模块 6 | API 契约与 Mock

目标：用接口 Schema 驱动覆盖分析、断言和虚拟服务。

主要功能

- 导入 OpenAPI/Swagger 或流量记录，保存端点、请求和响应 Schema。
- 比较契约版本，识别破坏性变更并标记受影响用例。
- 按响应 Schema 生成 Mock 数据，支持状态码、延迟、错误和条件匹配。
- Mock 配置绑定契约版本；未验证规则不得用于发布门禁环境。

设计理由：解决接口变化晚发现和外部依赖阻塞测试；契约与 Mock 同源可以减少两套定义漂移。

6.3.4 输入、输出与依赖

关系	内容
输入	已批准 DesignTestCase、Assertion、API Contract、数据与 Locator
输出	版本化 TestCase/TestSuite、可执行断言、数据快照、Mock 配置
上游	Test Design、Workspace、Platform Settings
下游	Execution、Quality Intelligence、CI/CD

6.3.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-TA-001	P0	用例与套件	TestCase CRUD、版本追溯、套件编排、CSV 导出和评审结果入库。
QF-FR-TA-002	P0	断言管理	维护正式用例或设计用例的结构化断言及验证状态。
QF-FR-TA-003	P1	Locator 资产与自愈	维护主备定位器、健康度和经审批的自愈候选。
QF-FR-TA-004	P1	测试数据	导入、合成、参数化和脱敏测试数据。
QF-FR-TA-005	P1	契约与 Mock	导入 API Contract、查看差异并配置延迟/故障模拟。

6.4 Execution（测试执行）

编排、派发和观测 API/UI 等测试任务，并收集结果、日志、截图和 Trace。

阶段	主要用户	核心对象	完成结果
V1	测试工程师、自动化工程师、Runner 运维人员	ExecutionPlan、ExecutionTask、ExecutionResult、RunnerNode	执行状态与结果证据可重放；HTTP 成功不等同于测试业务通过。

6.4.1 业务痛点

- 用例在不同机器和环境运行结果不一致，缺少环境、资产和 Runner 的完整快照。
- API/UI/性能等执行方式各自维护脚本和日志，调度、重试和状态语义不统一。
- 执行日志、截图、Trace 和断言结果分散，失败后难以还原现场。
- Runner 失联、任务取消和超时容易产生孤儿进程或重复执行。
- HTTP 请求成功、脚本退出码为 0 等技术信号容易被误当成业务测试通过。

6.4.2 设计思路与关键取舍

- 分离控制面与执行面：平台负责计划、状态和证据，Runner/Skill 负责受控物理执行。
- 每次任务锁定用例、套件、环境、数据、Skill 和 Runner 能力版本，保证可重放。
- 采用显式异步状态机与幂等触发，取消、重试、超时和恢复均产生新 attempt，不覆盖历史。
- 实时通道只负责观测和轻量打标；深度 RCA 交给 QI，避免执行引擎承担模型诊断。

6.4.3 子模块详解

子模块 1 | 执行计划与触发

目标：定义何时、在哪个环境、用哪些资产执行测试。

主要功能

- 创建计划并选择 TestSuite/TestCase 版本、环境、Runner 标签和并发策略。
- 支持手工、Cron、Webhook 和 CI/CD 触发，关键 POST 强制幂等键。
- 执行前校验资产状态、Secret 授权、环境可达性和 Runner 能力。
- 生成不可变 ExecutionPlan 快照并记录触发人、来源和参数。

设计理由：解决手工拼命令和每次运行条件不同；计划负责声明意图，任务负责一次具体实例。

子模块 2 | 任务队列与状态机

目标：管理一次执行从排队到终态的完整生命周期。

主要功能

- 维护 Pending/Queued/Running/Completed/Failed/Cancelled 等合法迁移。
- 拆分任务、用例和步骤粒度状态，汇总不能覆盖原始结果。
- 支持优先级、租户配额、并发限制、公平调度和取消。
- 重复请求返回同一任务或明确冲突；非法迁移由服务端拒绝。

设计理由：解决状态混乱、重复启动和资源争抢；显式状态机是前端、Runner 和 CI 联调的共同契约。

子模块 3 | Runner 注册与调度

目标：把任务路由到具备所需能力且健康的执行节点。

主要功能

- Runner 使用专用 Token 注册并上报 OS、驱动、Skill、标签和资源。
- 周期心跳更新 OnlineIdle/OnlineBusy/Offline/HeartbeatLost。
- 根据能力标签、负载、环境网络和租户策略选择节点。
- 失联后停止派发；是否重试或迁移按任务幂等性和版本能力决定。

设计理由：解决执行机器不可见和错误路由；能力匹配比简单轮询更能避免任务落到缺驱动的节点。

子模块 4 | 执行引擎与 Skill 调用

目标：以统一方式运行不同协议的测试步骤。

主要功能

- 把用例步骤转换为 Skill 输入，并校验 Schema、allowlist、超时和预算。
- 按步骤执行 HTTP、Playwright 等能力，收集断言和原生退出信息。
- Secret 只在调用前临时注入，Tool 输入输出和日志脱敏。
- 区分基础设施错误、脚本错误、断言失败和业务失败，禁止只看 HTTP/进程成功。

设计理由：解决各执行框架自成一套的问题；统一 Skill 契约让控制面保持稳定，同时允许扩展执行能力。

子模块 5 | 实时观测与证据

目标：在运行中提供进度，在完成后提供可审计证据。

主要功能

- 通过 WebSocket 推送日志、进度、步骤状态、CaseResult 和任务终态。
- 保存 stdout/stderr、HTTP 摘要、TraceId、截图、视频和性能数据引用。
- 支持 seq、心跳、断线续传和慢消费者控制；REST 提供最终快照。
- 运行期仅做阈值/模式打标，不生成最终 RCA 或修改资产。

设计理由：解决黑盒执行和断线丢日志；实时流服务人类观察，持久证据服务复盘与 QI，两者职责不同。

子模块 6 | 异常保护与恢复

目标：限制失败影响并保证任务可以安全终止或重放。

主要功能

- 配置步骤/用例/任务超时、有限重试和指数退避。
- 取消时停止未开始节点，终止可控进程并执行清理钩子。
- 检测孤儿任务、重复结果和 Runner 失联，保存现场后进入明确终态。
- 重放创建新 attempt 并引用旧任务，不覆盖失败证据。

设计理由：解决无限重试、脏环境和失败证据丢失；fail-closed 与 attempt 模型比原地重跑更可审计。

6.4.4 输入、输出与依赖

关系	内容
输入	ExecutionPlan、TestCase/Suite 快照、环境、数据、Skill、SecretRef
输出	ExecutionTask、ExecutionResult、日志/Trace/截图、实时事件
上游	Workspace、Test Asset、Skill Center、Platform Settings、CI/CD
下游	Quality Intelligence、CI/CD 门禁、Knowledge

6.4.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-EX-001	P0	计划与任务	创建执行计划，支持手工、定时和 CI 触发。
QF-FR-EX-002	P0	Runner 调度	注册 Runner、心跳、能力标签、并发配额和任务派发。
QF-FR-EX-003	P0	执行与结果	运行 API/UI 用例，记录用例粒度状态、证据和错误。
QF-FR-EX-004	P0	实时观测	通过 WebSocket 推送进度和日志，支持断线续传。
QF-FR-EX-005	P1	恢复与保护	超时、重试、取消、清理和故障转移按版本能力启用。

6.5 Quality Intelligence（质量智能）

对执行产物进行统计、失败归因、聚类、覆盖与发布风险分析。

阶段	主要用户	核心对象	完成结果
V1 基础 / V2 增强	测试工程师、测试负责人、研发负责人	QualityAnalysisReport、FailureAnalysisDetail、FailureCluster、CoverageReport	分析结论附证据、置信度与版本；AI 分类不冒充已确认缺陷。

6.5.1 业务痛点

- 传统报告只给 Pass/Fail，工程师仍需手工拼接日志、Trace、截图和代码变更。
- 同一根因导致大量失败，重复排查和重复提单掩盖真正影响范围。
- Flaky Test 与产品缺陷混在一起，导致门禁误阻或团队逐渐忽略失败。
- AI 给出的根因和修复建议缺少证据、版本和外部 oracle，容易产生虚假确定性。
- 覆盖率和风险分数若没有分母、算法和来源，无法复算也不能支撑发布决策。

6.5.2 设计思路与关键取舍

- QI 只消费 Execution 的不可变证据并生成只读分析产物，不回写篡改原始结果。
- 采用多证据印证：日志、Trace、截图、历史、变更和环境信息分别保留来源。
- AI 结论默认是候选分类和建议；关键 RCA、缺陷提单和发布门禁需确定性规则或人工确认。
- 把报告、失败明细、聚类、Coverage 和风险拆分，避免一个总分掩盖不确定性。

6.5.3 子模块详解

子模块 1 | 质量报告与指标

目标：汇总执行结果并提供可追溯的质量视图。

主要功能

- 展示总数、通过、失败、跳过、不稳定、耗时和趋势。
- 每个汇总数可下钻到 ExecutionResult 和原始证据。
- 区分任务失败、用例失败、基础设施异常和分析失败。
- 报告生成异步化并保存算法、数据窗口和版本。

设计理由：解决只有图表没有证据的问题；可下钻和版本化使报告可以被复核而非一次性展示。

子模块 2 | 失败证据归一化

目标：把不同 Runner 和 Skill 的失败现场转换为统一分析输入。

主要功能

- 提取异常类型、关键日志、调用链、HTTP、截图、环境和时间线。
- 脱敏 Secret、Token 和个人信息，保留证据哈希与原始位置。
- 识别证据缺失、时间不一致和 Runner 上报异常。
- 形成 FailureAnalysisDetail，供 RCA、聚类和人工排查共用。

设计理由：解决日志格式碎片化；先归一化再使用 AI 可以减少 Prompt 偶然性和重复解析。

子模块 3 | 根因分析与修复建议

目标：基于多源证据生成可解释的候选根因。

主要功能

- 区分 ProductBug、ScriptIssue、EnvIssue、DataIssue、DependencyIssue 等候选类型。
- 列出支持证据、反证、置信度、未知项和建议的下一步验证。
- 检索历史知识但标明命中来源和相似度；过期知识不得无提示使用。
- 人工确认 manualVerified 后才能把候选诊断当作已确认结论。

设计理由：解决 AI 一句话定责和错误建议；证据/反证/未知项结构要求模型展示可核查依据。

子模块 4 | 失败聚类与缺陷建议

目标：把相同或相关根因的失败聚合成可操作问题。

主要功能

- 按错误签名、Trace、受影响服务、时间和语义相似度形成候选聚类。
- 显示聚类代表失败、成员、差异和合并置信度。
- 人工可拆分、合并、命名和确认；变更保留聚类版本。
- 生成缺陷标题、复现、影响和证据包；写 Jira 前必须人工授权。

设计理由：解决重复提单和级联失败噪音；候选聚类加人工校正比模型直接建 Bug 更安全。

子模块 5 | 趋势、稳定性与 Flaky

目标：识别长期质量变化和不稳定测试。

主要功能

- 按用例、服务、分支、环境和版本统计失败率、波动和耗时。
- 通过受控重跑和时序证据识别 Flaky 候选，区分测试/环境/产品原因。
- 维护隔离建议、责任人和复验条件，禁止自动永久忽略。
- 统计 MTTR 等指标时声明起止事件和缺失数据。

设计理由：解决偶发失败干扰门禁和趋势失真；Flaky 是需要治理的状态，不是删除失败的理由。

子模块 6 | Coverage 与风险决策

目标：把覆盖缺口与质量证据转为可解释的决策输入。

主要功能

- CoverageReport 声明覆盖对象、分母、分子、缺口和 gate verdict。
- 结合变更范围、历史失败、关键路径和执行结果形成风险信号。
- 风险输出包含特征贡献、数据窗口和不确定性，不使用个人画像做不可解释评分。
- 向 CI/CD 提供确定性门禁输入；AI 推荐不能直接覆盖规则或放行。

设计理由：解决百分比和风险星级不可复算；显式算法与证据可以让发布人理解并质疑结论。

6.5.4 输入、输出与依赖

关系	内容
输入	ExecutionResult、日志/Trace/截图、Coverage 来源、Git 变更、Knowledge
输出	QualityAnalysisReport、FailureAnalysisDetail、FailureCluster、CoverageReport、候选 RCA
上游	Execution、Workspace、Knowledge
下游	CI/CD 门禁、Test Design 反馈、Knowledge 审核、外部缺陷系统

6.5.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-QI-001	P0	基础报告	展示执行通过率、失败分布、趋势与证据链接。
QF-FR-QI-002	P0	失败分析	关联日志、Trace、截图和历史知识形成候选原因。
QF-FR-QI-003	P0	覆盖报告	输出具有明确分母、分子和来源的 CoverageReport。
QF-FR-QI-004	P1	失败聚类与 RCA	聚合相同根因失败，并由金标准集和人工标注评估。
QF-FR-QI-005	P2	风险预测	结合历史质量和变更范围提供解释性风险信号，不直接代替发布人。

6.6 Skill Center（技能中心）

将 Playwright、HTTP、校验器等原子能力封装为受治理的可调用 Skill。

阶段	主要用户	核心对象	完成结果
V1 内置 / V2 开放	测试平台开发者、Skill 作者、平台管理员	Skill、SkillInvocation	发现/安装/静态 PASS 不等于真实调用通过；未在 allowlist 的 Skill 不可运行。

6.6.1 业务痛点

- API、UI、性能和安全测试框架接口不同，Agent 与执行引擎容易绑定具体实现。
- 自定义脚本依赖、权限和输出格式不可控，直接运行会带来越权、外联和供应链风险。
- “Skill 已安装”经常被误认为“能力可用”，缺少真实调用、版本和评测证据。
- 不同工作空间需要不同配置和启用状态，但不能修改全局内置 Skill 影响其他租户。

6.6.2 设计思路与关键取舍

- Skill 定义为单一、可复用、契约化的原子能力；Agent 决策、Skill 执行，两者不合并。
- Manifest 声明输入输出、依赖、权限、Must Not 和版本；调用前做 Schema 与 allowlist 校验。
- 内置 Skill 全局共享，启用态和配置按 Workspace 保存，避免租户互相影响。
- 自定义 Skill 先准入、后沙箱运行，并通过 eval 和真实调用证明能力，不以静态检查代替。

6.6.3 子模块详解

子模块 1 | 能力目录与工作空间启用

目标：让用户发现能力，同时把可见、已安装、已启用和可调用区分开。

主要功能

- 浏览官方、租户自定义和第三方 Skill 的版本、协议、作者、依赖和状态。
- 按 Workspace 启用/停用 Skill 并保存私有配置，不修改全局内置记录。
- 显示安装、结构校验、eval、真实调用和审批的不同证据状态。
- 升级前展示兼容性、权限变化和受影响 Agent/Workflow。

设计理由：解决“显示在列表就算可用”的误判；分层状态使发现、配置、验证和授权各自有证据。

子模块 2 | Manifest、注册与版本

目标：用机器可读契约管理 Skill 生命周期。

主要功能

- 导入包或编辑 Manifest，声明 ID、版本、协议、输入输出 Schema、依赖和入口。
- 校验命名、SemVer、引用、文件清单和依赖锁定。
- 版本不可原地覆盖；升级、停用和回滚保留历史调用引用。
- 禁止注册未声明网络、Secret 或外部写入需求的执行入口。

设计理由：解决脚本接口口口相传和升级破坏；Manifest 是 Agent、Runner 和审计共享的契约。

子模块 3 | 准入与评测

目标：在 Skill 进入可调用状态前验证结构、安全和业务行为。

主要功能

- 执行静态检查、依赖扫描、Schema 正反例和权限差异审查。
- 运行 evals/Golden fixtures，记录通过率、误判、性能和环境。
- 至少完成一次宿主真实调用，验证发现、路由、参数和结果。
- 高风险能力需双人审批；评测失败或缺失时不得 Active。

设计理由：解决 checker PASS 被误当成业务正确；多层准入把静态可解析、可执行和业务可信分开。

子模块 4 | 调用路由与参数映射

目标：把 Agent/Execution 请求安全地转换为具体 Skill 调用。

主要功能

- 根据协议、能力标签、版本、Workspace 启用态和 Runner 兼容性选择 Skill。
- 严格校验输入 Schema，拒绝模型臆造字段和未声明参数。
- 只从授权上下文注入环境变量与 SecretRef，记录脱敏输入输出哈希。
- 路由失败返回明确原因，不自动换用权限更大的能力。

设计理由：解决 Agent 与框架耦合和参数漂移；路由层允许底层能力替换，同时保持权限和契约稳定。

子模块 5 | 运行沙箱与资源控制

目标：限制自定义能力对平台和外部环境的影响。

主要功能

- 限制文件目录、进程、CPU、内存、时长、并发和包来源。
- 网络默认拒绝，只开放声明并获批的目标和协议。
- Secret 按调用最小注入，不写磁盘、不回显、不进入异常堆栈。
- 越权、超时、循环和异常外联立即终止并形成安全事件。

设计理由：解决插件等同任意代码执行的风险；硬隔离比仅靠 Prompt 或开发约定更可靠。

子模块 6 | 调用记录与分析

目标：提供能力级可靠性、成本和错误证据。

主要功能

- 记录调用人/Agent、Workspace、版本、Runner、耗时、状态和 ErrorCode。
- 区分路由、校验、沙箱、依赖、执行和业务错误。
- 统计 P50/P95、错误率、超时、资源使用和版本回归。
- 日志脱敏并设置保留期；不保存隐藏思维链或 Secret。

设计理由：解决 Skill 故障只能看 Runner 原生日志；统一调用记录能支撑容量、升级和故障归因。

6.6.4 输入、输出与依赖

关系	内容
输入	Manifest、Skill 包、调用请求、Workspace 配置、Runner 能力
输出	版本化 Skill、调用结果、SkillInvocation、评测与审计证据
上游	Platform Settings、Workspace、Agent Center、Execution
下游	Agent Workflow、Execution、Quality Intelligence

6.6.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-SK-001	P0	内置 Skill	提供 API Testing 与 Web Playwright 等固定能力及调用日志。
QF-FR-SK-002	P0	契约与版本	每个 Skill 声明输入、输出、Must Not、版本和评测入口。
QF-FR-SK-003	P1	准入与发布	自定义 Skill 经静态检查、eval、权限审查和人工审批后启用。
QF-FR-SK-004	P1	运行沙箱	限制文件、网络、Secret、CPU、内存、超时和外部写入。

6.7 Agent Center（Agent 中心）

配置受权限约束的 Agent、Prompt 版本、工作流和可审计 Trace。

阶段	主要用户	核心对象	完成结果
Gate 0 / V2	测试工程师、AI 平台工程师、审批人	Agent、AgentPromptVersion、AgentSession、AgentTraceNode	生成者与验证者职责分离；Agent 无权批准自己的产物或修改权威源。

6.7.1 业务痛点

- AI 能力散落在各页面和 Prompt 中，模型、版本、工具权限与结果无法统一管理。
- 复杂任务让单个 Agent 自由规划容易循环、越权、跳过前置 Gate 或调用不存在的工具。
- 生成者自检自批形成同源偏差，所谓“AI 已验证”缺少独立性。
- 仅保存聊天文本无法重放工具调用、路由和失败现场，也难以评估版本回归。

6.7.2 设计思路与关键取舍

- Gate 0 先采用 1 个 Supervisor + 3 个受限 Subagent + 固定 DAG，不先建设通用多 Agent 平台。
- Agent 只承担职责和决策，所有物理动作通过 allowlist Skill；权限由机器配置强制，而不是 Prompt 自律。
- 生成、验证、审批三权分离；AI 可生成和验证，但无权做人的正式批准。
- Trace 记录决策摘要、证据和工具事件，不采集或展示隐藏思维链。

6.7.3 子模块详解

子模块 1 | Agent 注册与 Prompt 版本

目标：管理专用 Agent 的职责、模型、提示词和生命周期。

主要功能

- 维护 Agent ID、职责、模型、参数、状态、允许 Skill 和禁止动作。
- Prompt 版本支持草稿、测试、灰度、激活和回滚，不原地覆盖。
- 配置变更显示权限差异、评测结果和受影响 Workflow。
- V0 只允许 qf-supervisor、qf-requirement-ir、qf-test-design、qf-independent-verifier。

设计理由：解决 Prompt 散落和角色漂移；专用 Agent 比“万能 Agent”更容易限定权限和建立评测集。

子模块 2 | Workflow 与任务编排

目标：用可验证 DAG 约束 Agent 和 Skill 的执行顺序。

主要功能

- 定义节点、依赖、输入输出、超时、重试、终止和人工检查点。
- Supervisor 负责预检、派发、终止和汇总，最大委派深度为 1。
- PASS 才进入下一节点；REJECT/ERROR/TIMEOUT 停止，BLOCK_HUMAN 暂停。
- V0 使用版本化 JSON DAG；真实闭环稳定后再评估可视化编辑器。

设计理由：解决模型跳步骤和无限委派；固定 DAG 先证明业务闭环，比先造通用编排平台风险更低。

子模块 3 | 会话与上下文管理

目标：为一次 Agent 任务锁定目标、资料、权限和预算。

主要功能

- 创建 runId/sessionId，保存用户目标、输入清单、敏感级别和禁止动作。
- 锁定术语、领域、Skill、Workflow、模型和 Prompt 版本/哈希。
- 上下文只按最小必要加载，来源变化时终止或从受影响节点重放。
- 超时、取消和失败保存现场；重放创建新 attempt。

设计理由：解决上下文混用和运行不可复现；锁定版本可区分模型问题、资料变化和 Skill 回归。

子模块 4 | Tool 授权与调用控制

目标：确保 Agent 只能调用其职责内的 Skill。

主要功能

- 调用前同时校验 allowedSkills、输入 Schema、网络、Secret、预算和前置产物。
- 自然语言相似度只用于发现候选，不构成授权。
- 同一 Skill/输入禁止循环；同参失败最多重试 1 次。
- 正式资产写入、外部系统写入和发布动作必须人工批准并由应用服务执行。

设计理由：解决 Agent 因模型提示而越权；前置强制校验比事后审计更能降低损害。

子模块 5 | Trace、异常与重放

目标：提供可审计而不过度暴露模型内部信息的运行记录。

主要功能

- 记录节点、Agent、Skill、版本、输入输出哈希、时间、裁决和 Finding Code。
- 展示结构化决策摘要、证据引用和路由结果，不展示隐藏 CoT。
- 异常保存脱敏 Tool 输入输出、错误和停止原因。
- 支持按 runId 重放到指定节点，但不得覆盖原运行证据。

设计理由：解决“模型为什么这样做”无法核查和失败无法复现；事件 Trace 比完整聊天转储更可控。

子模块 6 | 评估与反馈

目标：衡量 Agent 是否真正改善业务结果，而不只统计调用成功。

主要功能

- 统计路由正确率、人工修改率、误放/误阻、成本、超时和越权尝试。
- 按 Agent/Prompt/模型/Skill 版本比较 Golden Set 与真实抽样。
- 人工反馈关联具体 Trace 节点和最终处置，不把点赞当作唯一质量指标。
- 未建立 oracle 前不得用执行成功率替代业务正确率。

设计理由：解决 Token/成功率看板掩盖业务错误；版本化评测才能支持灰度、回滚和持续改进。

6.7.4 输入、输出与依赖

关系	内容
输入	用户目标、权威资料、Agent/Workflow 配置、Skill 注册表
输出	AgentSession、AgentTraceNode、草稿产物、机器裁决与人工路由
上游	Platform Settings、Knowledge、Skill Center、AI 治理
下游	Test Design、Execution、Quality Intelligence、人工审批服务

6.7.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-AG-001	P0	Agent 配置	管理模型、Prompt 版本、状态和 Skill allowlist。
QF-FR-AG-002	P0	会话与 Trace	记录决策摘要、证据、工具输入输出哈希和路由，不采集隐藏思维链。
QF-FR-AG-003	P0	Gate 0 治理	采用 1 Supervisor + 3 受限 Subagent + 13 个 Skill 的固定 DAG。
QF-FR-AG-004	P1	工作流编排	在验证最小闭环后提供可视化 DAG 与受控版本发布。

6.8 Knowledge Center（知识中心）

管理 RAG 知识、经验模式和接口/UI/性能基线，为 Agent 提供可追溯上下文。

阶段	主要用户	核心对象	完成结果
V1	测试工程师、知识审核人、平台管理员	KnowledgeItem、KnowledgeVector、BaselineItem	检索结果有来源和版本；过期、冲突或未审核知识不得无提示参与高风险裁决。

6.8.1 业务痛点

- 需求、规范、故障经验和修复记录散落，人员离开或时间推移后知识难以复用。
- 单纯向量检索可能召回过期、错误或跨空间资料，增加 AI 幻觉和数据泄露风险。
- 接口 Schema、视觉截图和性能阈值分散在脚本中，断言缺少统一基线。
- AI 自动抽取的知识若直接发布，会把错误诊断和偶然经验长期固化。
- 知识缺少生命周期和冲突治理，RAG 内容越积越多但质量下降。

6.8.2 设计思路与关键取舍

- 分离原始资料、索引分块、经验知识和测试基线，避免一个“大向量库”承载所有语义。
- 采用关键词 + 向量 + 过滤的混合检索，并强制 Workspace、状态、版本和权限过滤。
- AI 抽取只产生 Draft/PendingReview，人工审核后才能 Published 并用于高风险流程。
- 所有检索结果返回来源、版本、分块位置和相关度；使用者可以判断证据质量。

6.8.3 子模块详解

子模块 1 | 多源知识导入

目标：接收并治理进入知识系统的原始资料。

主要功能

- 支持 PDF、Word、Markdown、OpenAPI、故障记录和经验卡片导入。
- 执行类型、大小、恶意内容、编码、来源和重复哈希检查。
- 保存原文、来源、版本、Workspace、权限和解析状态。
- 解析失败不发布空知识；重新导入产生新版本。

设计理由：解决资料散落与来源不明；保留原文和元数据才能让后续索引与回答可追溯。

子模块 2 | 分块、索引与混合检索

目标：把已准入资料转换为可过滤、可解释的检索索引。

主要功能

- 按标题、段落、Schema 和语义边界分块，避免固定长度切断含义。
- 生成关键词索引与向量索引，记录 Embedding 模型和版本。
- 查询时先做租户、Workspace、状态、时间和类别过滤，再进行混合排序。
- 返回 Top-K 结果、来源位置、相关度和索引版本。

设计理由：解决纯向量检索误召回和跨租户风险；结构过滤与混合排序更适合技术术语和精确字段。

子模块 3 | 团队经验与 Testing Pattern

目标：把已确认的缺陷和解决方法沉淀为可复用模式。

主要功能

- 维护场景、触发条件、症状、根因、验证步骤、修复和适用边界。
- 关联已确认 RCA、Bug、TestCase 和代码/配置版本。
- 允许评级、标签、订阅和废弃；引用旧版本时显示警告。
- Test Design 和 QI 使用时必须保留知识项 ID 与版本。

设计理由：解决经验只存在于个人脑中或聊天记录；结构化模式比无边界的总结更容易复用和验证。

子模块 4 | 测试基线库

目标：集中管理自动断言所依赖的 API、视觉和性能基线。

主要功能

- 保存 API Response Schema、UI 截图/区域、性能水位及版本。
- 基线变更先生成候选 Diff，显示受影响断言和用例。
- 按环境和分支区分基线，禁止测试环境数据覆盖生产邻近基线。
- 审批、启用、回滚和过期均记录审计。

设计理由：解决基线散落和静默更新导致“测试永远通过”；候选变更与审批保证基线不是被测结果自我修改。

子模块 5 | AI 抽取与人工审核

目标：从已确认活动中生成知识草稿并控制发布质量。

主要功能

- 从已关闭缺陷、人工确认 RCA 和用例修正中提取 Pattern 草稿。
- 显示来源、置信度、重复项、冲突和待补字段。
- 审核人可修改、合并、驳回或发布，并记录理由。
- 生成 Agent 不得批准自己的知识；未审核草稿不进入关键 RAG。

设计理由：解决自动学习把错误永久化；人机分工让 AI 降低整理成本，但知识权威仍由人确认。

子模块 6 | 维护、冲突与生命周期

目标：持续控制知识的新鲜度、重复度和访问范围。

主要功能

- 检测近似重复、相互矛盾、来源失效和长期未使用知识。
- 提供合并、降级、归档、删除和重新评审流程。
- 权限或 Workspace 变化时重建相关索引并验证隔离。
- 统计检索命中、人工相关性、过期率和误用反馈。

设计理由：解决 RAG 越用越乱；生命周期治理比单纯增加文档数量更能提高检索质量。

6.8.4 输入、输出与依赖

关系	内容
输入	项目文档、OpenAPI、已确认 RCA/Bug、用例修正、基线候选
输出	KnowledgeItem、KnowledgeVector、BaselineItem、带来源检索结果
上游	Workspace、QI、Test Asset、人工审核
下游	Test Design、Agent、QI、Execution 断言

6.8.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-KN-001	P0	知识导入与检索	导入多源文档，执行分块、索引和混合检索。
QF-FR-KN-002	P0	知识审核	AI 抽取内容先形成草稿，人工审核后发布。
QF-FR-KN-003	P0	基线管理	维护 API Schema、性能和视觉基线的版本与状态。
QF-FR-KN-004	P1	经验闭环	从已确认问题和修复记录抽取可复用 Testing Pattern。

6.9 CI/CD Center（持续集成与交付）

连接外部流水线，触发测试、执行 TIA、计算质量门禁并回传结果。

阶段	主要用户	核心对象	完成结果
V1 基础 / V2 增强	测试工程师、研发工程师、发布负责人	PipelineConfig、PipelineRun、QualityGate	门禁计算确定性、可重放；AI 建议不能自行放行发布。

6.9.1 业务痛点

- 每次提交执行全量回归耗时过长，但只跑少量冒烟又可能漏掉受影响路径。
- 质量门禁只看总体通过率，关键用例、Flaky 和基础设施失败混在一起。
- 测试结果、RCA 和证据与 PR/MR 脱节，研发需要切换系统并手工定位。
- Webhook、重试和重复事件缺少统一处理，容易重复启动昂贵任务。
- AI 风险建议若直接放行或阻断，会把不可解释模型判断引入发布控制。

6.9.2 设计思路与关键取舍

- QualityForge 作为 CI 平台的测试与门禁适配层，不重复建设 Jenkins/GitLab/GitHub 的流水线引擎。
- TIA 只负责推荐和形成动态回归集，必须保留变更到用例的映射证据，并支持全量回退。
- QualityGate 使用版本化确定性规则计算；AI 只提供风险解释和补测建议。
- Webhook 验签、去重、幂等和回调重试独立治理，避免外部系统波动污染业务状态。

6.9.3 子模块详解

子模块 1 | 流水线与 Webhook 集成

目标：安全接收外部变更事件并关联工作空间。

主要功能

- 配置 Jenkins/GitLab/GitHub、项目、分支、事件类型和回调地址。
- 验证签名、timestamp、nonce 和重放窗口；Secret 不回显。
- 按 provider/eventId/commit 幂等去重并保存原始事件摘要。
- 映射到 Workspace、PipelineConfig 和目标 ExecutionPlan。

设计理由：解决重复触发和伪造事件；把集成做成适配层可以保持核心执行模型不依赖某个 CI 厂商。

子模块 2 | 流水线运行与状态回传

目标：协调外部流水线与内部执行任务的生命周期。

主要功能

- 创建 PipelineRun 并关联 commit、分支、触发人和 ExecutionTask。
- 回传排队、运行、完成、失败和取消状态及报告链接。
- 外部回调失败采用有限重试和死信，不修改已经计算的门禁结论。
- 支持从外部取消并保留已完成执行证据。

设计理由：解决 CI 只知道“接口调用成功”却不知道测试终态；独立运行实体能跨异步边界追踪。

子模块 3 | 测试影响分析 TIA

目标：根据变更选择足够且可解释的回归集合。

主要功能

- 解析 Git Diff 中的文件、模块、类、接口和配置变更。
- 映射 BusinessRule、APIContract、TestCase 和历史失败关系。
- 输出直接/间接影响、选中/跳过用例、理由和置信度。
- 证据不足、核心路径或规则冲突时自动回退全量/保守集合。

设计理由：解决全量太慢与冒烟漏测之间的矛盾；可解释选择和保守回退比追求最高裁剪率更安全。

子模块 4 | 质量门禁

目标：用确定性规则把测试证据转换为发布控制信号。

主要功能

- 按分支/环境配置关键用例、通过率、严重失败、Coverage 和基础设施规则。
- 规则版本化，计算时锁定输入报告与隔离清单。
- 输出 Passed/Warned/Blocked/Quarantined 及逐条命中原因。
- 人工例外需记录审批人、原因、有效期和补偿措施。

设计理由：解决单一通过率掩盖关键失败；确定性、版本化门禁可以重放并接受审计。

子模块 5 | Flaky 隔离与复验

目标：降低已确认不稳定用例对主流水线的噪音，同时继续治理。

主要功能

- 只对有证据和责任人的 Flaky 候选启用临时 Quarantine。
- 隔离用例继续执行并单独报告，不计入指定门禁指标。
- 设置到期、复验条件和自动提醒；到期未处理恢复门禁影响或阻断。
- 禁止把未知失败自动标成 Flaky。

设计理由：解决团队因偶发失败绕过全部门禁；临时隔离保留可见性和修复责任。

子模块 6 | 结果反馈与通知

目标：把执行结果和可操作证据带回开发协作界面。

主要功能

- 在 PR/MR 展示门禁、失败摘要、Coverage 缺口和报告链接。
- 候选 RCA 与修复建议明确标记 AI 生成和人工确认状态。
- 通知渠道按严重级别、订阅和去重策略发送。
- 任何外部评论、状态写入或发布放行都需要授权 Scope。

设计理由：解决测试平台与研发工作流割裂；把摘要送到 PR 可以缩短反馈，但完整证据仍留在权威报告。

6.9.4 输入、输出与依赖

关系	内容
输入	Webhook/Git Diff、Execution 状态、QI 报告、QualityGate 规则
输出	PipelineRun、动态回归集、门禁结论、PR/通知反馈
上游	Workspace、Platform Settings、Execution、QI
下游	外部 CI、Git PR/MR、发布负责人

6.9.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-CI-001	P0	Webhook 与运行	接收 Jenkins/GitLab/GitHub 触发并关联执行任务。
QF-FR-CI-002	P0	质量门禁	按版本化规则计算 Passed/Warned/Blocked 等结果。
QF-FR-CI-003	P1	TIA	基于 Git Diff 和资产映射选择受影响用例，并保留选择证
QF-FR-CI-004	P1	结果回传	将进度、报告链接和诊断摘要回传外部系统。

6.10 Platform Settings（平台设置）

承载租户级用户偏好、LLM 网关、Runner、Secret 和外部集成配置。

阶段	主要用户	核心对象	完成结果
V1	平台管理员、租户管理员、Runner 运维人员	User、Role、LlmConfig、RunnerNode、WorkspaceSecret、IntegrationConfig	租户隔离、密钥不可回显、管理动作有审计；默认拒绝未授权外联。

6.10.1 业务痛点

- LLM、Runner、Secret、用户和外部集成配置散落在环境变量、脚本和各业务模块。
- 多模型供应商接口、配额和故障处理不同，业务模块直接调用会形成厂商耦合。
- Runner 能力和健康不可见，任务失败后难以判断是资产问题还是执行节点问题。
- 凭证明文或可回显会进入日志、Prompt 和截图，造成严重泄露。
- 旧设计强调单一测试工程师，无法支撑租户管理员、审批人和发布负责人等权限。

6.10.2 设计思路与关键取舍

- Platform Settings 是控制面，不承载测试业务资产；它向其他模块提供身份、模型、Runner、Secret 和集成能力。
- 所有配置按 tenant/workspace 分层，平台默认、租户策略和空间覆盖有明确优先级。
- LLM 通过统一网关抽象供应商，Runner 通过能力注册抽象物理节点，业务模块不绑定具体实现。
- Secret 采用双信封加密和最小临时注入；读取接口永不返回明文。

6.10.3 子模块详解

子模块 1 | 用户、角色与偏好

目标：管理身份、租户成员、职责权限和个人体验设置。

主要功能

- 维护用户、租户、成员状态、角色和通知偏好。
- 采用 RBAC 权限并结合资源归属校验；同一人可以承担多个角色。
- 角色变更、禁用、登录和高风险管理动作写审计。
- 默认 Workspace、语言、时区和通知属于个人偏好，不改变业务权限。

设计理由：解决单一用户假设和审批权限缺失；职责权限与个人偏好分离可避免设置项意外扩大授权。

子模块 2 | LLM 模型网关

目标：为 Agent 和 AI 功能提供统一、可治理的模型访问。

主要功能

- 配置供应商、模型、端点、权重、TPM、状态和租户级 API Key。
- 提供连通性、延迟、限流、重试、熔断和受控级联降级。
- 记录 Token、费用、模型/Prompt 版本和请求结果，不记录敏感原文。
- 业务模块按能力和策略请求模型，不直接读取供应商密钥。

设计理由：解决模型厂商耦合、密钥散落和成本不可见；统一网关也为降级和审计提供单一控制点。

子模块 3 | Runner 节点池

目标：管理执行节点的注册、能力、健康和生命周期。

主要功能

- 生成短期注册命令/Token，Runner 上报 OS、驱动、Skill、标签和资源。
- 监控心跳、版本、CPU/内存和依赖健康，支持停用与维护状态。
- Runner Token 只允许专用接口，轮转和撤销不影响用户 Token。
- 向 Execution 提供可调度节点视图，不直接决定业务任务结果。

设计理由：解决执行基础设施黑盒化；节点池统一登记能力，但调度决策仍由 Execution 根据任务上下文完成。

子模块 4 | Secret 与凭证

目标：集中保护 LLM、Git、CI、数据库和测试账号等敏感信息。

主要功能

- 保存名称、类型、作用域、所有者、过期时间和加密载荷。
- 使用 KEK/DEK 双信封加密，支持轮转、撤销和重新加密。
- 读取接口只返回掩码和元数据；解密仅在授权调用的短生命周期内发生。
- 审计谁在何时为哪个任务使用了哪个 SecretRef，不记录明文值。

设计理由：解决凭证复制和日志泄露；集中管理使轮转与吊销可控，SecretRef 让业务数据无需接触明文。

子模块 5 | 外部集成配置

目标：管理 Jira、Git、CI、LDAP/SSO 和通知渠道的连接与授权。

主要功能

- 配置 provider、端点、OAuth/API Key/SSH SecretRef 和 Scope。
- 提供连通性测试、回调地址、Webhook 验签和权限预览。
- 按 Workspace 授权具体项目/仓库/渠道，禁止全局凭证被任意空间使用。
- 停用或授权变化时通知依赖模块并阻止新写入。

设计理由：解决各模块重复保存外部凭证和权限过大；统一连接管理减少集成漂移并明确作用域。

子模块 6 | 审计、配额与平台治理

目标：提供跨模块的安全、成本和管理证据。

主要功能

- 记录登录、权限、Secret、模型、Runner、Skill 和集成配置变更。
- 配置租户/Workspace 的模型、执行、存储和并发配额。
- 对越权、密钥使用异常、配额超限和节点异常生成告警。
- 审计数据只读、脱敏并有保留期；高风险导出需审批。

设计理由：解决管理动作不可见和资源失控；统一审计与配额避免每个业务模块各造一套不一致机制。

6.10.4 输入、输出与依赖

关系	内容
输入	用户/租户、模型配置、Runner、Secret、外部集成和平台策略
输出	认证授权上下文、模型路由、可用 Runner、SecretRef、IntegrationConfig、审计事件
上游	企业身份源、KMS/Vault、模型供应商、Runner、外部工具
下游	Workspace、Agent、Skill、Execution、Knowledge、CI/CD

6.10.5 功能需求摘要

需求 ID	优先级	能力	要求
QF-FR-PS-001	P0	用户与角色	维护账号偏好、租户归属及 RBAC。
QF-FR-PS-002	P0	LLM 网关	配置模型、路由、健康、配额和成本，不显示明文密钥。
QF-FR-PS-003	P0	Runner 管理	注册、心跳、健康检查、标签和停用。
QF-FR-PS-004	P0	Secret 与集成	双信封加密保存凭证，按最小权限临时注入。

7. AI、Agent 与 Skill 治理

7.1 自治等级

等级	含义	允许动作
L0	无 AI / 确定性自动化	校验、格式转换、确定性规则和人工操作
L1	AI 建议	生成建议或草稿；人确认后生效
L2	受控自动执行	在 allowlist、预算、阈值和回滚范围内执行，并保留人工检查点
L3	高度自治	仅在充分评测、监控和可回滚场景显式启用；不作为 Gate 0 默认

7.2 最小 Agent 拓扑

Agent	职责	允许的 Skill 范围	禁止事项
qf-supervisor	分类、预检、派发、终止、汇总	plan-orchestrator	不生成业务资产、不审批、不发布
qf-requirement-ir	需求评审包和 IR 草稿	需求质量、规则、领域、状态、编译	不改原始需求、不批准自产 IR
qf-test-design	策略、场景、用例、候选断言	策略、场景、用例、断言生成	不执行测试、不修改已验证 IR
qf-independent-verifier	独立验证和人工路由	IR、断言、Coverage、HITL	不修复被审资产、不代替人工批准

7.3 权限与触发

- 默认文件只读；仅可写单次运行目录。docs、Skill 原型和 Golden 样例禁止直接写。
- 默认 deny_all 网络、无 Secret、无外部写入；临时放宽必须有目标、用途、过期时间和审批记录。
- Skill 触发必须同时满足目标、前置产物、Schema、allowlist、版本、预算、权限和人工检查点。
- 同参失败最多重试 1 次；超时、循环、越权、Schema 错误均 fail-closed。
- 生成者与验证者必须是不同角色配置；切换 Prompt 不构成独立复核。
- 不记录隐藏思维链，只记录决策摘要、证据引用、工具调用和脱敏后的输入输出。

8. 数据、集成与接口需求

8.1 数据与追踪

- 所有业务资源必须带 tenant/workspace 归属；服务端根据认证上下文校验，不信任客户端单独声明。
- 正式资产保存版本、创建者/审批人、时间、状态、输入输出哈希和 SourceTrace。
- DesignTestCase 是设计草稿，审批迁移后生成 TestCase；不得用一个模糊状态机覆盖两阶段语义。
- CoverageReport 必须声明覆盖对象、分母、分子、缺口和算法版本。
- 删除、归档、重放和模型/Prompt/Skill 升级均保留审计证据。

8.2 接口与集成

类别	要求
REST	统一 /api/v1、camelCase JSON、Bearer/API Key/Runner Token、统一错误语义、幂等和分页。
WebSocket	Execution、AI 诊断、Agent Trace、Workspace 扫描四类通道，支持 seq、心跳、续传和租户隔离。
CI/CD	Webhook 验签、防重放、状态回传和门禁计算；外部写入必须显式授权。
知识与文档	支持文件和 URL 导入；来源、版本、校验和与解析状态可追踪。
LLM	通过统一网关路由；密钥双信封加密、日志脱敏、超时、重试、熔断和成本统计。

9. 非功能需求

质量属性	V1 基线	后续能力 / 验证方式
性能	50+ API 并发；FCP < 3.0s；AI 用例生成 P95 < 20s	压测报告需记录数据量、Runner、模型、网络与资源利用率
安全	逻辑多租户隔离；AES-256-GCM；RBAC；日志脱敏	RLS/越权测试、Secret 不回显、上传安全、依赖和沙箱审计
可靠性	模型调用 1 次重试；Runner 中断标记失败	V2 引入心跳故障转移、断点和熔断；故障注入验证
可观测性	结构化 JSON 日志、基础 Metrics	requestId/runId/traceId 全链传播；Agent 工具与成本指标
一致性	单服务本地事务；关键写操作幂等	跨服务 Saga 仅在实际拆分后启用；补偿可重放
可访问性	键盘操作、焦点、标签、颜色对比和错误提示	核心流程按 WCAG 2.2 AA 做自动+人工检查
隐私	最小收集、目的限制、数据分级和保留期	敏感文档、Prompt、日志、截图和模型传输路径审计

10. 路线图与版本范围

版本	目标	主要范围	明确延后
Gate 0 / V0	验证可信链	需求质量→IR→规则→场景→用例→断言→Coverage→人工评审	正式运行时平台、外部写入、生产自治
V1（1–3 月）	跑通单节点闭环	Workspace、需求/用例、正式资产、API/UI 单机执行、基础报告、内置 Skill/Agent、基础 RAG/Webhook	分布式 Runner、自定义 Skill、多 Agent 自治
V2（4–6 月）	智能增强与整合	契约扫描、Locator 自愈、分布式执行、深度 RCA、自定义 Skill、Workflow、TIA/门禁	超大规模与企业市场
V3（7–12 月）	企业级规模化	弹性 Runner、视觉测试、风险预测、Skill 市场、多 Agent、持续反馈、PR 回写	需由真实使用和资源重新立项

开发准入判断

B-1/B-2/B-3 的契约地基已在 2026-07-31 与 2026-08-04 的变更中补齐，但尚待独立复审；B-4/B-5/B-6 仍影响验收可信度。建议只在 Sprint 0 完成角色裁决、OpenAPI operationId、AI 阈值/评测计划和跨源复验后进入全平台开发。

11. 验收与发布门禁

11.1 项目级验收样例

AC-E2E-01 可信链

Given 一份包含可测规则、角色和验收条件的需求已导入；**When** 完成质量 Gate、IR、场景、设计用例、断言和覆盖分析；**Then** 每个正式用例可追溯到需求或规则；任一硬 Gate 非 PASS 时不得发布 TestCase。

AC-SEC-01 租户隔离

Given 用户 A 与用户 B 属于不同租户；**When** 用户 A 使用合法 Token 请求用户 B 的任一资源 ID；**Then** 返回 403 或不泄露存在性的 404，响应与日志不包含资源内容。

AC-AI-01 独立复核

Given 生成 Agent 已产出 IR/用例/断言草稿；**When** 生成 Agent 尝试批准或验证自己的产物；**Then** 系统在执行前拒绝并记录越权；必须路由到独立验证与人工审批。

AC-EX-01 执行证据

Given 执行任务包含有效用例、环境和 Runner 能力；**When** 任务完成、失败、取消或 Runner 中断；**Then** 状态转换合法，结果与证据可查询；HTTP 2xx 不被解释为业务 PASS。

AC-CI-01 门禁重放

Given 相同版本规则、输入报告和隔离清单；**When** 重复计算质量门禁；**Then** 得到相同结论和原因代码，并可追溯至规则版本。

11.2 开发完成定义

- 需求 ID 已映射到页面/API/实体/测试；不允许只以代码完成或接口 2xx 作为验收。
- OpenAPI 可解析，operationId 唯一，请求/响应/错误/鉴权/幂等/分页契约有正反测试。
- 状态机跨 PRD、领域、API、DB 一致；非法迁移被服务端拒绝。
- AI 输出通过独立 oracle、Golden Set 或人工抽检；模型、Prompt、Skill 和数据版本可追踪。
- 关键 NFR 有实测环境、数据、结果和限制说明；安全与租户隔离无阻断缺陷。
- 用户验收记录与遗留风险由有权负责人签署；发布/回滚方案可执行。

12. 风险、依赖与未决事项

ID	事项	影响	建议处置
DEC-01	角色模型仍与旧总纲单用户表述冲突	权限和审批无法实现	正式裁决多角色 RBAC，并同步术语/API/DB
DEC-02	164 个 OpenAPI 操作全部缺 operationId	SDK、契约测试和追踪不稳定	Sprint 0 按域补唯一 operationId
DEC-03	AI 阈值缺统一校准集	PASS/REVIEW 边界不可解释	集中配置阈值，建立风险分层 Golden Set
DEC-04	LLM 输出缺外部 oracle	执行成功无法证明业务正确	确定性 checker + 双人标注 + 盲测
DEC-05	收益目标缺基线	无法证明 ROI	先采现状样本，冻结口径后再设目标
DEC-06	全平台范围大于单团队短周期能力	基础设施挤压价值验证	Gate 0/V1 单体优先，使用证据决定 V2/V3
DEC-07	审计报告与后续修订存在时间差	旧裁决可能被误读为当前状态	完成独立复审并发布新的统一裁决

附录 A. 权威源与追踪规则

领域	权威源	用途
导航与当前状态	54-项目需求资料总索引.md	阅读顺序、资料状态、现行统计
术语	03-术语表Glossary.md	模块、实体、状态、AI 概念与废弃别名
实体	04-核心领域模型.md	字段、关系和状态机
模块需求	10–19 号 PRD	功能、流程、页面和模块边界
AI 治理	21-AI介入点降级矩阵.md	L0–L3、默认自治和降级
Agent/Skill	55-Agent与Skill运行治理SOP.md	权限、allowlist、触发、审批和运行
API	32-API规范.yaml	REST 路径、Schema、鉴权和响应
DB	33-数据库Schema设计.md	表、约束、索引和租户隔离
认证/实时	34-认证授权设计.md / 36-WebSocket实时契约.md	RBAC/RLS/Secret 与实时通道
审计	docs/05-audits/current/	缺陷、成熟度和准入裁决

变更顺序：术语或实体先更新 03/04；状态同步模块 PRD、04、32、33；AI 阈值只在 21 裁决；新增正式实体必须同时进入 03、04、32、33，并在复审后更新本整合 PRD。